Домашняя работа по лекции 4

1. Выбор веб-приложения и определение используемых протоколов

Приложение: YouTube

Используемые протоколы:

Уровень	Протокол	Назначение
Прикладной	HTTP/3 (поверх QUIC)	Загрузка веб-страниц, API, метаданные видео
Прикладной	HTTP/2 (поверх TCP) - резервный вариант	Для браузеров/сетей без поддержки QUIC
Прикладной	DASH / HLS (поверх HTTP)	Адаптивная потоковая передача видео
Транспортный	QUIC (поверх UDP)	Основной транспорт для HTTP/3
Транспортный	TCP	Резервный транспорт для HTTP/2, а также для служебных соединений
Транспортный	UDP	Передача медиапотоков в реальном времени (YouTube Live), QUIC
Прикладной	WebSocket (поверх TCP)	Чат в прямых трансляциях, уведомления
Прикладной	DNS (поверх UDP)	Разрешение имён

Как это работает на практике:

- При открытии сайта браузер устанавливает соединение по QUIC/HTTP/3. Если сеть блокирует UDP - откат на HTTP/2/TCP.
- Видео передаётся через DASH: плеер скачивает сегменты видео по HTTP, адаптируя качество под пропускную способность.
- В прямых эфирах и чате используется WebSocket и UDP для медиапотоков с низкой задержкой.

2. Обоснование выбранного стека и альтернативы

QUIC/HTTP/3 как основа:

- Снижение задержки: 1-RTT при первом соединении и 0-RTT при повторном - критично для быстрой загрузки страниц.
- Устранение Head-of-Line Blocking: при потере пакета один поток не блокирует остальные. Для видеосервиса, где параллельно загружаются десятки ресурсов, это даёт ощутимый прирост.
- Мобильность соединения: пользователь переключается с Wi-Fi на LTE - соединение QUIC сохраняется, видео не прерывается. Для TCP это означало бы разрыв и повторное рукопожатие.
- Встроенное шифрование (TLS 1.3): безопасность «из коробки», метаданные скрыты от промежуточных узлов.

HTTP (DASH/HLS) для видео:

- Позволяет использовать CDN и кеширование - видеохранилища YouTube распределены по тысячам серверов.
- Адаптивный битрейт: плеер выбирает качество сегмента в зависимости от скорости сети.
- Работает поверх любого транспорта (QUIC или TCP), что обеспечивает совместимость.

WebSocket для чата:

- Постоянное соединение с минимальными накладными расходами на заголовки - эффективнее, чем постоянные HTTP-запросы.
- Двусторонняя коммуникация в реальном времени.

UDP для live-медиа:

- Для прямых трансляций важна минимальная задержка, а небольшие потери допустимы.

Какие альтернативы могли бы быть и почему не подходят

Альтернатива	Почему не подходит
Чистый TCP для всего	Head-of-Line Blocking: потеря одного сегмента останавливает доставку всех последующих данных. Для страницы с десятками параллельных ресурсов это заметно замедляет загрузку. Плюс 3-RTT на установку соединения (TCP + TLS).
Чистый UDP для видео без надстройки	Нет гарантии доставки, нет управления перегрузкой - при перегрузке сети качество катастрофически падает, возможны артефакты. Нужна собственная реализация надёжности (что и делает QUIC).

Альтернатива	Почему не подходит
gRPC для публичного API	Браузеры не поддерживают gRPC напрямую (нужен gRPC-Web с прокси). Для публичного видеосервиса с миллиардами клиентов это лишняя сложность. HTTP/JSON универсален.
MQTT/AMQP для доставки видео	Это протоколы обмена сообщениями, не предназначенные для потоковой передачи больших объёмов медиаданных.
FTP для передачи файлов	Устарел, не поддерживает адаптивность, плохо работает через NAT/прокси, нет шифрования по умолчанию.

3. Преимущества и ограничения перехода на QUIC/HTTP/3

Преимущества

1. Ускорение загрузки на 20-30% за счёт 0-RTT и устранения HOL-блокировки. Для видеосервиса это означает более быстрый старт воспроизведения и отзывчивый интерфейс.
2. Улучшение работы в нестабильных сетях (мобильный интернет, общественный Wi-Fi): QUIC лучше переносит потери пакетов, так как каждый поток независим.
3. Мобильность соединения: переключение Wi-Fi - LTE без разрыва - критично для мобильных пользователей.
4. Встроенное шифрование TLS 1.3: упрощает архитектуру безопасности, скрывает метаданные от провайдеров и промежуточных узлов.
5. Гибкость управления перегрузкой: алгоритмы можно обновлять на уровне приложения, не дожидаясь обновления ОС.

Ограничения

1. Блокировка UDP в некоторых сетях: корпоративные файрволы, гостиничные и аэропортовые сети часто блокируют UDP. Требуется fallback на TCP/HTTP/2 - то есть инфраструктура должна поддерживать оба стека.
2. Нагрузка на CPU: шифрование всего трафика и обработка QUIC в user space требуют больше вычислительных ресурсов, чем TCP в ядре ОС. Для серверов YouTube это существенные затраты.
3. Сложность отладки: трафик зашифрован, включая служебные заголовки - сетевым инженерам сложнее диагностировать проблемы.
4. Зрелость экосистемы: не все CDN, балансировщики и инструменты мониторинга полностью поддерживают QUIC. Требуется обновление инфраструктуры.

5. Проблемы с кешированием на промежуточных узлах: поскольку метаданные зашифрованы, прокси-серверы не могут эффективно кешировать контент на транспортном уровне.

4. Приложение, для которого UDP подходит лучше TCP

Пример: многопользовательская онлайн-игра.

Почему UDP лучше

1. Минимальная задержка критична. В шутере решение о попадании принимается за миллисекунды. TCP с его подтверждениями, повторными передачами и HOL-блокировкой добавил бы задержку, которая делает игру неиграбельной. UDP отправляет пакет и не ждёт подтверждения.
2. Потеря пакета допустима. Если потерялся пакет с позицией игрока в конкретный момент - не страшно: следующий пакет (через 16 мс при 60 FPS) уже содержит актуальную позицию. Повторная передача устаревших данных бессмысленна и только увеличивает задержку. В TCP потерянный сегмент блокировал бы доставку всех последующих, пока не будет восстановлен.
3. Устаревшие данные не нужны. В TCP при потере пакета данные доставляются в правильном порядке, но с задержкой - игрок увидит «застывшую» картинку, а затем рывок. В UDP игра просто пропускает устаревший кадр и показывает актуальный - плавность важнее полноты.
4. Собственная реализация надёжности там, где нужно. Разработчики игр реализуют поверх UDP выборочную надёжность: например, критические события подтверждаются и повторяются, а позиции - нет. Это гибче, чем жёсткая надёжность TCP.

Что было бы с TCP

- Задержка из-за рукопожатия при подключении.
- HOL-блокировка: потеря одного пакета - «фриз» всей игры до восстановления.
- Избыточные повторные передачи устаревших данных.
- Невозможность тонкой настройки: TCP не различает «важные» и «неважные» данные.